



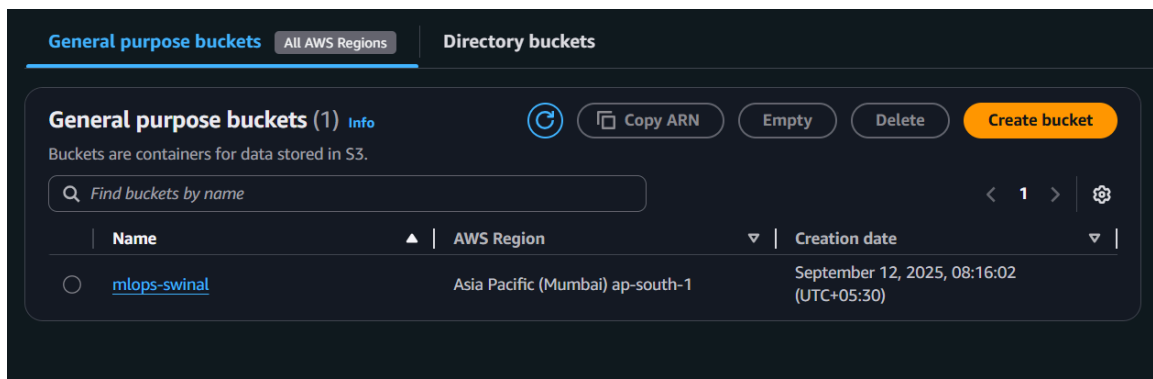
IAM Role

Date	@September 11, 2025
Multi-select	IAM IAM-Roles Policies Users
Select	Lab-3
Status	Not started

Roles in AWS are primarily used for **service-to-service connections**. Instead of manually managing user credentials, roles provide a secure way to allow one AWS service to access another. Let's walk through an example of how an EC2 instance can securely access an S3 bucket using an IAM role.

Step 1: Create an S3 Bucket Inside the Root Account

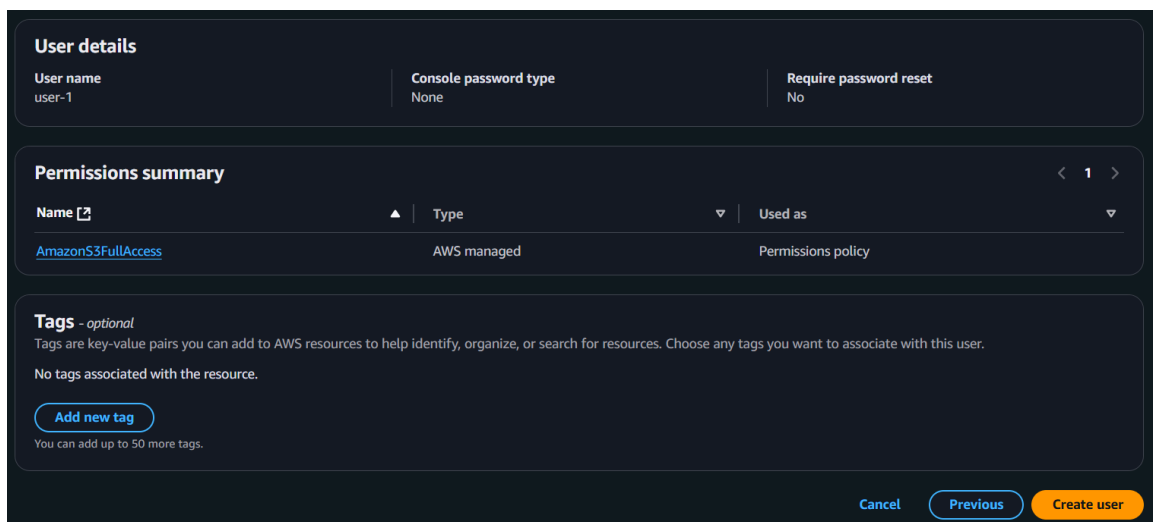
- We'll create a general-purpose bucket with default settings.
- The only requirement is that the bucket name must be globally unique.



Step 2: Why Do We Need Roles?

To understand this, let's first create an IAM user:

- Create a user named **user-1** and attach the managed policy **AmazonS3FullAccess**.



- Next, generate security credentials for this user in the form of **Access Keys**:
 - Click **Create access key** → **Command Line Interface (CLI)** → select **user-1** → Next → Done.
 - Download the **.csv** file or simply click **Done**.

Retrieve access keys [Info](#)

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key	Secret access key
AKIATJH3JLLL6GLNLM6K	***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#) [Done](#)

Step 3: Launch an EC2 Instance

- Launch an EC2 instance with default settings.

Instance summary for i-0c4995c0e191b034c (test-ec2) [Info](#)

Updated less than a minute ago

Instance ID i-0c4995c0e191b034c	Public IPv4 address 3.108.184.101 open address	Private IPv4 addresses 172.31.39.71
IPv6 address -	Instance state Running	Public DNS ec2-3-108-184-101.ap-south-1.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-39-71.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-39-71.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendation s. Learn more
Auto-assigned IP address 3.108.184.101 [Public IP]	VPC ID vpc-04e69030a6e05c83c	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-0dddf4221da6eb0ae	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:226012781271:instance/i-0c4995c0e191b034c	

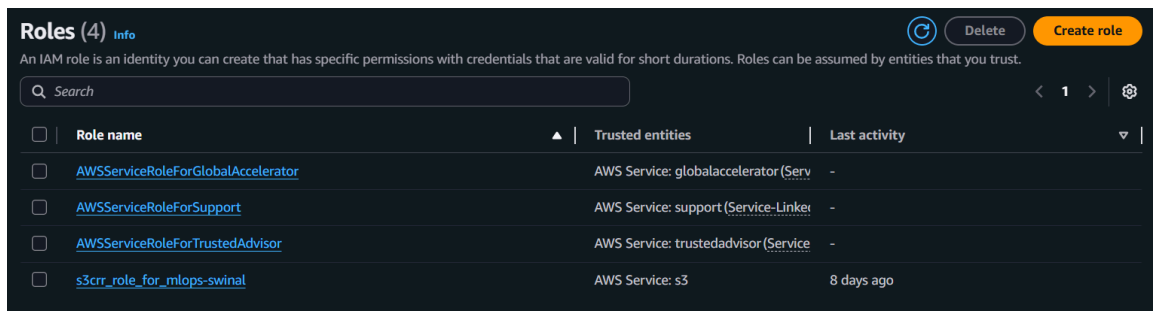
- Now, assume this EC2 instance needs to access the S3 bucket. If we try accessing it directly, we get an error:

- Inside `.aws/credentials`, both the **Access Key** and **Secret Access Key** are stored in plain text. This is a major security risk.

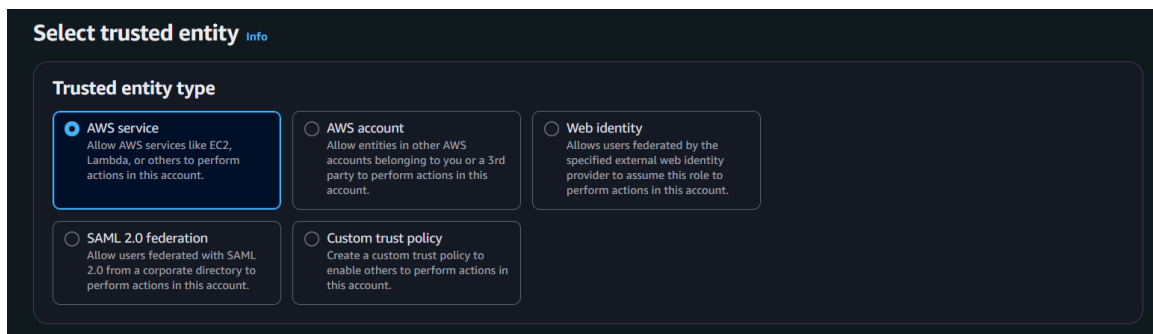
👉 To solve this, we use **IAM Roles**.

Step 5: Create an IAM Role

- Go to IAM and click **Create Role**.



- Choose the trusted entity type. Since this is a service-to-service connection (EC2 → S3), select **AWS Service** → **EC2**.



Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
EC2

Choose a use case for the specified service.

Use case

- ☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.
- ☐ **EC2 Role for AWS Systems Manager**
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.
- ☐ **EC2 Spot Fleet Role**
Allows EC2 Spot Fleet to request and terminate Spot Instances on your behalf.
- ☐ **EC2 - Spot Fleet Auto Scaling**
Allows Auto Scaling to access and update EC2 spot fleets on your behalf.
- ☐ **EC2 - Spot Fleet Tagging**
Allows EC2 to launch spot instances and attach tags to the launched instances on your behalf.
- ☐ **EC2 - Spot Instances**
Allows EC2 Spot Instances to launch and manage spot instances on your behalf.
- ☐ **EC2 - Spot Fleet**
Allows EC2 Spot Fleet to launch and manage spot fleet instances on your behalf.
- ☐ **EC2 - Scheduled Instances**
Allows EC2 Scheduled Instances to manage instances on your behalf.

Cancel Next

- Attach the policy **AmazonS3FullAccess**.

Step 1 Select trusted entity
Step 2 **Add permissions**
Step 3 Name, review, and create

Add permissions info

Permissions policies (1/1087) info

Choose one or more policies to attach to your new role.

Filter by Type All types 26 matches

Policy name	Type	Description
☐ AmazonDMSRedshiftS3Role	AWS managed	Provides access to manage S3 settings...
☒ AmazonS3FullAccess	AWS managed	Provides full access to all buckets via t...
☐ AmazonS3ObjectLambdaExecutionRolePolicy	AWS managed	Provides AWS Lambda functions perm...

- Name the role **EC2-S3** and create it.

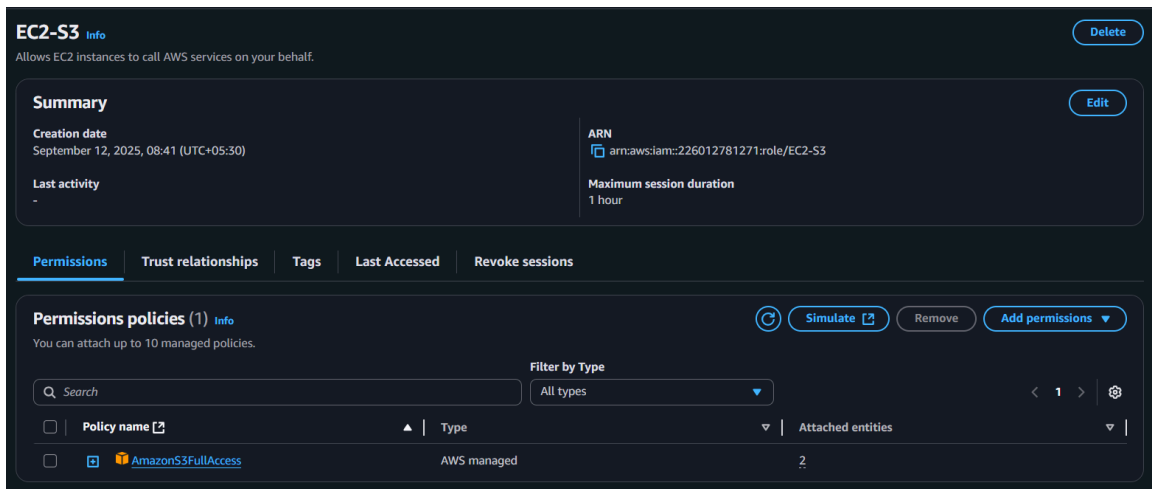
Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.
EC2-S3-Access
Maximum 64 characters. Use alphanumeric and '+', '@', '-', '_' characters.

Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.
Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: '_', '+', '@', '/', '[', ']', '#', '\$', '%', '^', '*', '=', '&', '<', '>', '–', '—', '‘', '’', '“', '”', '„', '‟', '‚', '‛', '”', '„', '‟', '“', '”', '„', '‟', '“', '”', '„', '‟', '“', '”', '„', '‟'

- Role created: **EC2-S3**.



Step 6: Attach the Role to the EC2 Instance

- First, remove the IAM user credentials from the EC2 instance:

```
cd .aws
ls
rm -rf *
ls
```

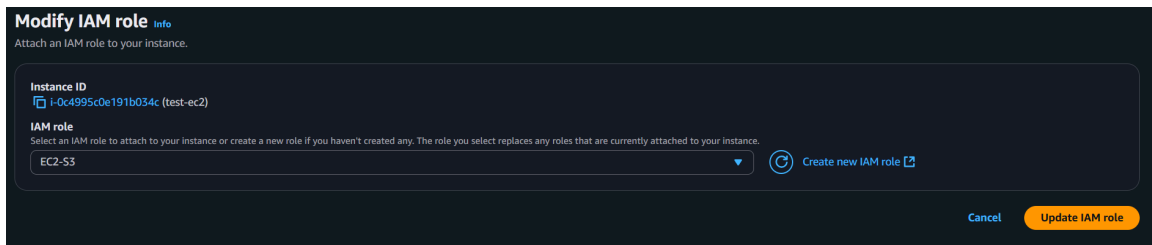
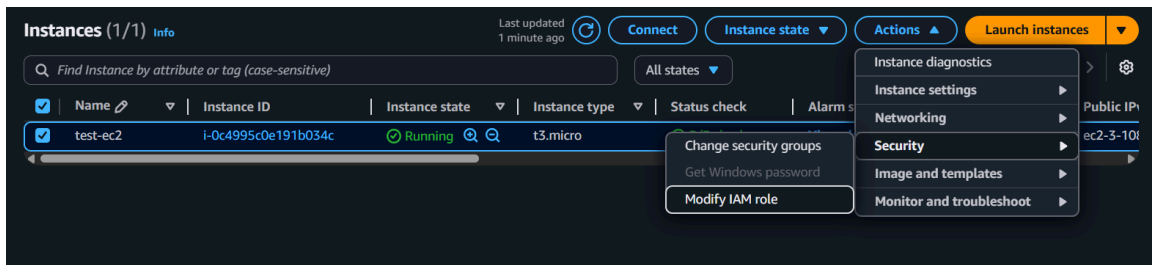
```
[root@ip-172-31-39-71 .aws]# ls
config  credentials
[root@ip-172-31-39-71 .aws]# rm -rf *
[root@ip-172-31-39-71 .aws]# ls
[root@ip-172-31-39-71 .aws]#
```

- At this point, the instance has no credentials, so accessing S3 fails:

```
aws s3 ls --profile user1
```

```
[root@ip-172-31-39-71 .aws]# aws s3 ls --profile user1
The config profile (user1) could not be found
[root@ip-172-31-39-71 .aws]#
```

- Now, attach the **EC2-S3** role to the instance:
 - Go to **EC2 Dashboard** → Select Instance → **Actions** → **Security** → **Modify IAM role**.
 - Select the role **EC2-S3** → **Update IAM role**.



- Now, retry accessing S3:

```
aws s3 ls
```

```
[root@ip-172-31-39-71 .aws]# cd ~
[root@ip-172-31-39-71 ~]# aws s3 ls
2025-09-12 02:46:03 mlops-swinai
[root@ip-172-31-39-71 ~]#
```

Step 7: How IAM Roles Improve Security

- Unlike IAM user credentials, **IAM Role credentials are not stored in** `.aws/credentials`.

```
[root@ip-172-31-39-71 ~]# cd .aws
[root@ip-172-31-39-71 .aws]# ls
[root@ip-172-31-39-71 .aws]#
```

- IAM Roles provide **temporary credentials** that are automatically rotated.
 - Validity ranges between **1 hour (minimum)** and **12 hours (maximum)**.
- You can verify the active identity using:

```
aws sts get-caller-identity
```

```
[root@ip-172-31-39-71 .aws]# aws sts get-caller-identity
{
  "UserId": "AROATJH3JLLI4MMVLHCR4:i-0c4995c0e191b034c",
  "Account": "226012781271",
  "Arn": "arn:aws:sts::226012781271:assumed-role/EC2-S3/i-0c4995c0e191b034c"
}
[root@ip-172-31-39-71 .aws]#
```


Conclusion

Using IAM roles is a much more **secure and scalable** way to connect AWS services compared to manually managing access keys. Roles ensure:

- No permanent credentials stored inside instances.
- Automatic rotation of temporary security credentials.
- Easy attach/detach from services without changing application code.

This makes IAM Roles the recommended approach whenever you need **service-to-service access** in AWS.